Based on this explanation, the CSRF token may sound very similar to the Origin header. After all, they both describe where a request originates. However, there are some key differences, as summarized in table C.1. These differences make it a good idea to use a CSRF token even if an Origin header is available. CSRF protection is especially important for simple requests, where there is no preflight to protect the server from invalid requests.

Table C.1 Differences between the Origin header and CSRF token

Origin header	CSRF token
Set by the browser	Set by the server
Value is in plain text	Value is encrypted
Can be guessed (and spoofed using tools like curl)	Cannot be guessed or spoofed
Only present on cross-origin requests (although some browsers, such as Chrome and Safari, include Origin headers on some same-origin requests)	Present on cross-origin and same-origin requests

The next section looks at how to implement CSRF protection for same-origin requests. The techniques here don't apply to CORS, but they're useful for getting an understanding of how CSRF protection works.

C.2 Implementing CSRF protection for same-origin requests

It may be easier to understand CSRF tokens by looking at a new example that isolates the core concepts of CSRF. Listing C.1 shows a simple web server that implements a CSRF token. Note that this is new server code, so you should put this in a new app.js file (but running it is the same as before—just run `node app.js`). The Express framework has middleware support for CSRF tokens. You can install this middleware (and its dependencies) by running the following command:

```
npm install express body-parser cookie-parser express-session csurf
```

Listing C.1 Example of implementing CSRF protection

```
var express = require('express');
var bodyParser = require('body-parser');
var cookieParser = require('cookie-parser');
var session = require('express-session');
var csrf = require('csurf');

var app = express();
app.use(bodyParser.urlencoded({
  extended: true
}));
app.use(cookieParser());
app.use(session({
  secret: 'CORSInAction',
  saveUninitialized: true,
  resave: true}));
app.use(csrf());
```

Creates new server and adds middleware components